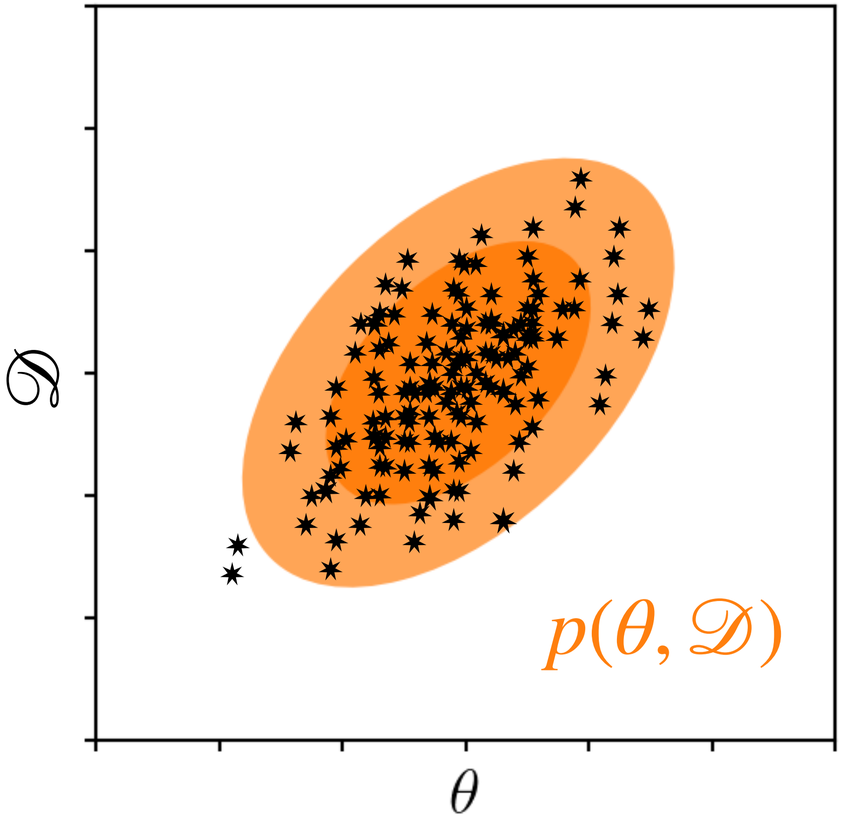
Lecture 21:
neural density estimation

November 4, 2026

with a prior $p(\theta)$ and F, we can sample the joint distribution $p(\theta, \mathscr{D})$

$$\theta' \sim p(\theta) \quad \mathscr{D}' \sim F(\theta')$$



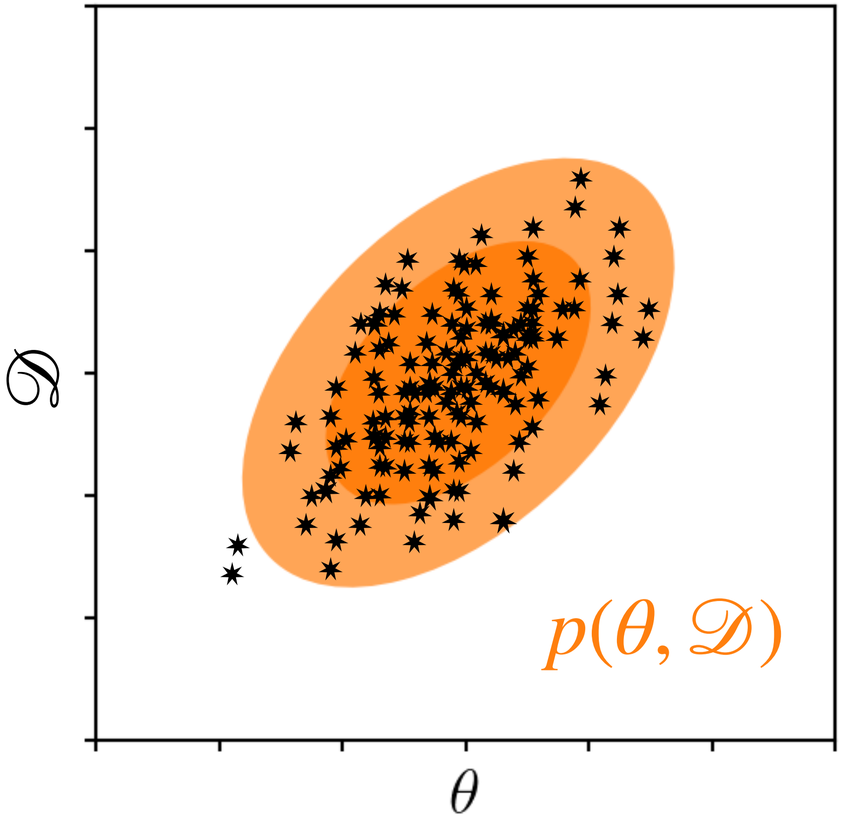
with a prior $p(\theta)$ and F, we can sample the joint distribution $p(\theta, \mathscr{D})$

$$\theta' \sim p(\theta) \quad \mathscr{D}' \sim F(\theta')$$

approximate bayesian computation

$$p(\theta \mid \mathscr{D}_{\text{obs}}) \approx p(\theta \mid \mathscr{D} \approx \mathscr{D}_{\text{obs}})$$

$$\approx p(\theta \mid \rho(\mathscr{D}_{\text{obs}}, \mathscr{D}) < \epsilon)$$



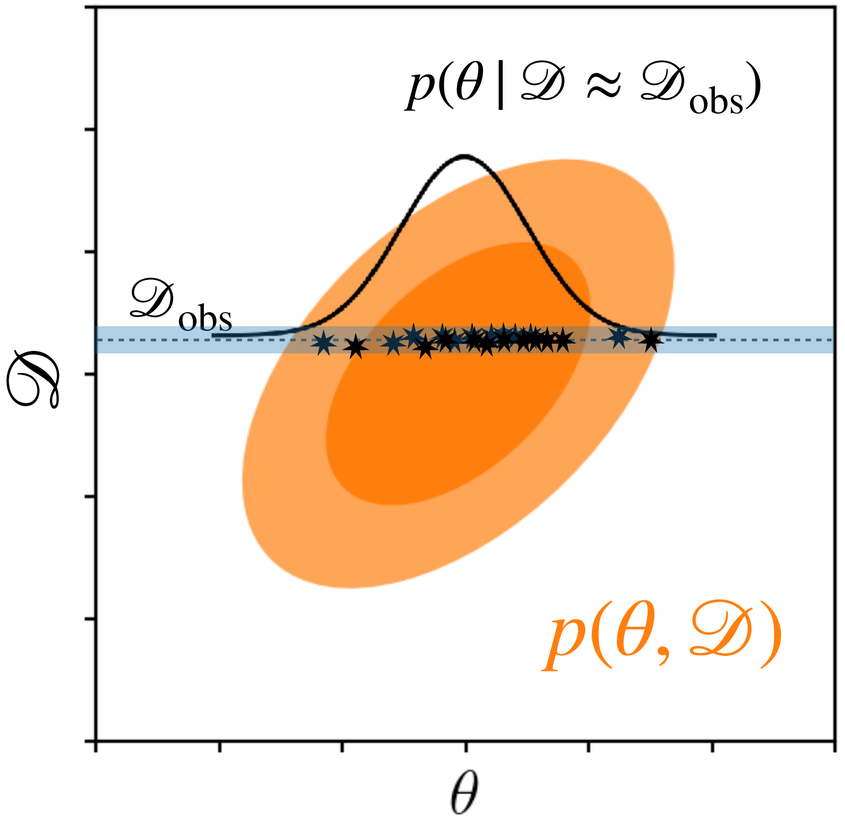
with a prior $p(\theta)$ and F, we can sample the joint distribution $p(\theta, \mathscr{D})$

$$\theta' \sim p(\theta) \quad \mathscr{D}' \sim F(\theta')$$

approximate bayesian computation

$$p(\theta \mid \mathscr{D}_{\text{obs}}) \approx p(\theta \mid \mathscr{D} \approx \mathscr{D}_{\text{obs}})$$

$$\approx p(\theta \mid \rho(\mathscr{D}_{\text{obs}}, \mathscr{D}) < \epsilon)$$



SBI with variational inference — Bayesian workflow

1. sample $\{(\theta', \mathscr{D}')\} \sim p(\theta, \mathscr{D})$ using simulator F

2. select $q_\phi(\theta, \mathscr{D})$ and optimize ϕ

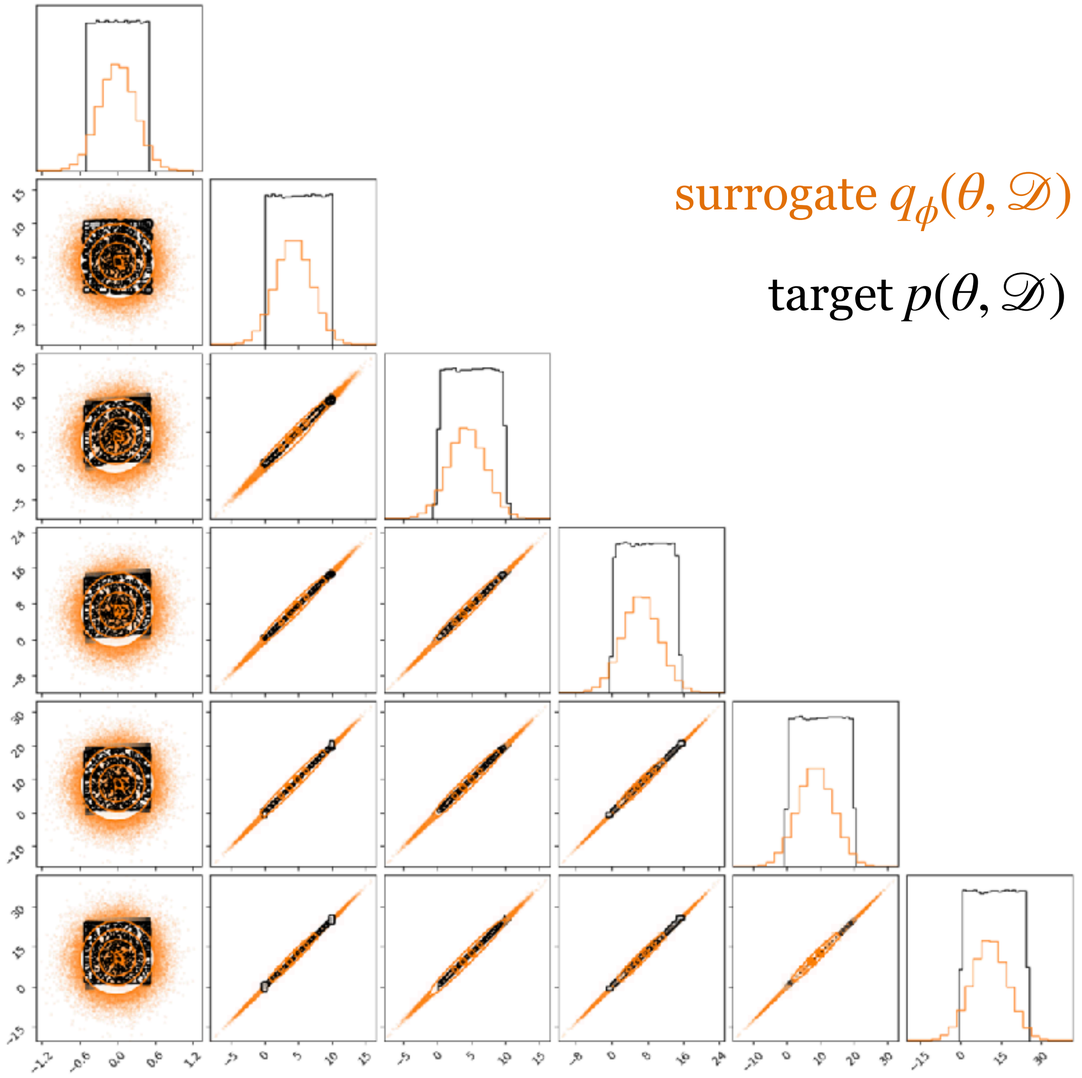
$$\min_\phi D_{\mathrm{KL}}\left(p(\theta, \mathscr{D}) \parallel q_\phi(\theta, \mathscr{D})\right) \approx \sum_{(\theta', \mathscr{D}') \sim p(\theta, \mathscr{D})} \left[-\log q_\phi(\theta', \mathscr{D}')\right]$$

3. ABC using surrogate q_ϕ

$$(\theta', \mathscr{D}') \sim q_\phi(\theta, \mathscr{D})$$

we can sample the joint distribution **without** running an expensive simulator

SBI with variational inference — variational approaches are sensitive to choice of surrogate

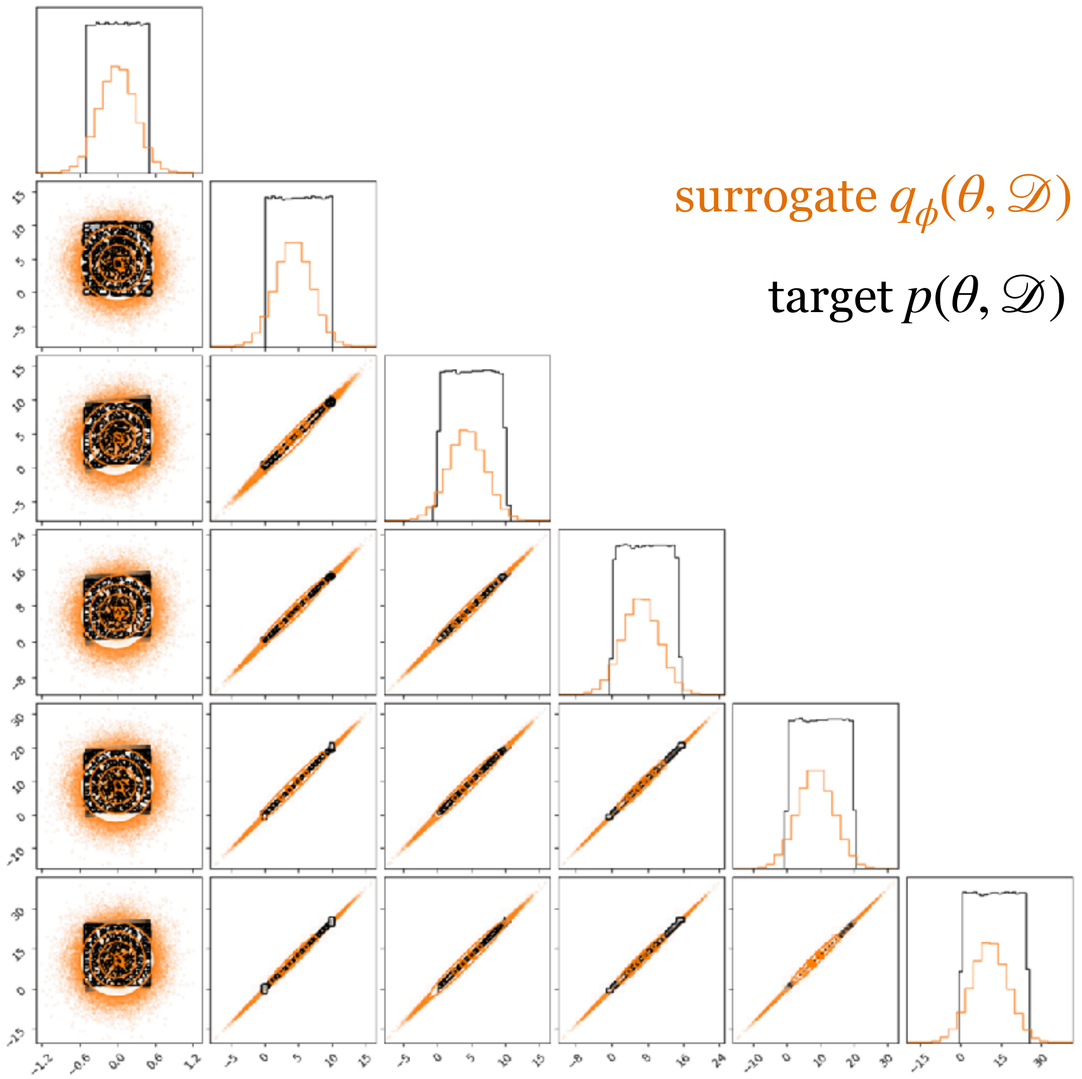


surrogate $q_\phi(\theta, \mathscr{D})$

target $p(\theta, \mathscr{D})$

see SBI jupyter notebook

SBI with variational inference — variational approaches are sensitive to choice of surrogate



surrogate $q_\phi(\theta, \mathscr{D})$

target $p(\theta, \mathscr{D})$

multivariate Gaussian

see SBI jupyter notebook

how can we **improve** SBI with variational inference?

how can we **improve** SBI with variational inference?

1. we want q_ϕ to be highly flexible to describe a wide range of target distributions

how can we **improve** SBI with variational inference?

1. we want q_ϕ to be highly flexible to describe a wide range of target distributions

2. we want q_ϕ to be easy to evaluate and differentiable w.r.t. ϕ

$$\min_\phi D_{\mathrm{KL}}\left(p(\theta, \mathscr{D}) \parallel q_\phi(\theta, \mathscr{D})\right) \approx \sum_{(\theta', \mathscr{D}') \sim p(\theta, \mathscr{D})} \left[-\log q_\phi(\theta', \mathscr{D}')\right]$$

how can we **improve** SBI with variational inference?

1. we want q_ϕ to be highly flexible to describe a wide range of target distributions

2. we want q_ϕ to be easy to evaluate and differentiable w.r.t. ϕ

$$\min_\phi D_{\mathrm{KL}}\left(p(\theta, \mathscr{D}) \parallel q_\phi(\theta, \mathscr{D})\right) \approx \sum_{(\theta', \mathscr{D}') \sim p(\theta, \mathscr{D})} \left[-\log q_\phi(\theta', \mathscr{D}')\right]$$

3. we want q_ϕ to be easy to sample

neural density estimation

neural density estimation

Neural density estimation is a parametric method for density estimation that uses neural networks to parameterize a density model. A *neural density estimator* is a neural network with parameters $\boldsymbol{\phi}$ that takes as input a datapoint $\mathbf{x}$ and returns a real number $f_{\boldsymbol{\phi}}(\mathbf{x})$ such that:

$$\int_{\mathbb{R}^D} \exp(f_{\boldsymbol{\phi}}(\mathbf{x}))\, \mathrm{d}\mathbf{x} = 1. \tag{2.29}$$

The above constraint is enforced by construction, that is, the architecture of the neural network is such that $\exp(f_{\boldsymbol{\phi}}(\mathbf{x}))$ integrates to 1 for all settings of $\boldsymbol{\phi}$ (I will discuss how this can be achieved in section 2.3). Since $\exp(f_{\boldsymbol{\phi}}(\mathbf{x}))$ meets all the requirements of a density function, the neural network can be used as density model $q_{\boldsymbol{\phi}}(\mathbf{x}) = \exp(f_{\boldsymbol{\phi}}(\mathbf{x}))$.

Papamakarios (2019)

neural density estimation can be *much* more flexible

Neural density estimation is a parametric method for density estimation that uses neural networks to parameterize a density model. A *neural density estimator* is a neural network with parameters $\boldsymbol{\phi}$ that takes as input a datapoint $\mathbf{x}$ and returns a real number $f_{\boldsymbol{\phi}}(\mathbf{x})$ such that:

$$\int_{\mathbb{R}^D} \exp(f_{\boldsymbol{\phi}}(\mathbf{x}))\, \mathrm{d}\mathbf{x} = 1. \tag{2.29}$$

The above constraint is enforced by construction, that is, the architecture of the neural network is such that $\exp(f_{\boldsymbol{\phi}}(\mathbf{x}))$ integrates to 1 for all settings of $\boldsymbol{\phi}$ (I will discuss how this can be achieved in section 2.3). Since $\exp(f_{\boldsymbol{\phi}}(\mathbf{x}))$ meets all the requirements of a density function, the neural network can be used as density model $q_{\boldsymbol{\phi}}(\mathbf{x}) = \exp(f_{\boldsymbol{\phi}}(\mathbf{x}))$.

Papamakarios (2019)

examples of neural density estimators

Masked Autoencoder for Distribution Estimation (MADE; *Germain et al. 2015*)
Masked Autoregressive Flow (MAF; *Papamakarios et al. 2017*)

autoregressive density estimation

autoregressive density estimation





Claude



same mathematical ideas as autoregressive language models

autoregressive density estimation — estimate some N-dim probability distribution $p(\mathbf{x})$

$$p(\mathbf{x}) =$$

autoregressive density estimation — estimate some N-dim probability distribution $p(\mathbf{x})$

$$
\begin{aligned}
p(\mathbf{x}) = \; & p(x_N \mid x_1, x_2, \ldots x_{N-2}, x_{N-1}) \times \\
& p(x_{N-1} \mid x_1, x_2, \ldots x_{N-2}) \times \\
& \ldots \\
& p(x_3 \mid x_1, x_2) \times \\
& p(x_2 \mid x_1) \times \\
& p(x_1)
\end{aligned}
$$

Chain rule

autoregressive density estimation — estimate some N-dim probability distribution $p(\mathbf{x})$

$$
\begin{aligned}
p(\mathbf{x}) = \; & p(x_N \mid x_1, x_2, \ldots x_{N-2}, x_{N-1}) \times \\
& p(x_{N-1} \mid x_1, x_2, \ldots x_{N-2}) \times \\
& \ldots \\
& p(x_3 \mid x_1, x_2) \times \\
& p(x_2 \mid x_1) \times \\
& p(x_1)
\end{aligned}
\quad \Big| \; \text{Chain rule}
$$

$$
p(\mathbf{x}) = \prod_{i=1}^{N} p(x_i \mid \mathbf{x}_{i-1:1})
$$

autoregressive density estimation — estimate some N-dim probability distribution $p(\mathbf{x})$

$$p(\mathbf{x}) = \prod_{i=1}^{N} p(x_i \mid \mathbf{x}_{i-1:1})$$

1-dimensional

autoregressive density estimation — estimate some N-dim probability distribution $p(\mathbf{x})$

$$p(\mathbf{x}) = \prod_{i=1}^{N} p(x_i \mid \mathbf{x}_{i-1:1})$$

describe using a **neural network**

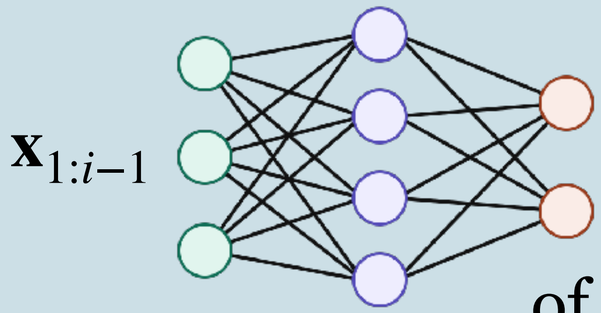
autoregressive density estimation — estimate some N-dim probability distribution $p(\mathbf{x})$

$$p(\mathbf{x}) = \prod_{i=1}^{N} p(x_i \mid \mathbf{x}_{i-1:1})$$

describe using a **neural network**

e.g.

$\mathrm{NN}(\mathbf{W}_i, \mathbf{b}_i, \mathbf{x}_{1:i-1})$



of 1D Gaussian

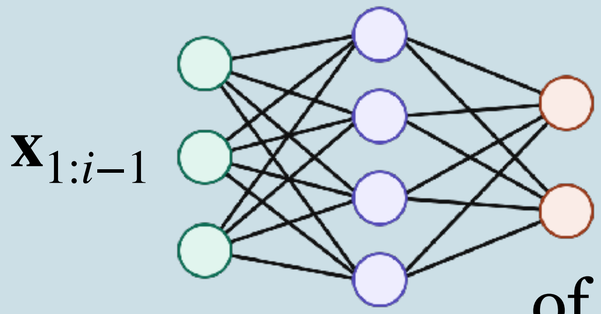
autoregressive density estimation — estimate some N-dim probability distribution $p(\mathbf{x})$

$$p(\mathbf{x}) = \prod_{i=1}^{N} p(x_i \mid \mathbf{x}_{i-1:1})$$

describe using a **neural network**

e.g.

$\mathrm{NN}(\mathbf{W}_i, \mathbf{b}_i, \mathbf{x}_{1:i-1})$



of 1D Gaussian

$p(x_i \mid \mathbf{x}_{1:i-1})$ is easy to sample

$$x_i' \sim \mathcal{N}\left(\mu_i(\mathbf{W}_i, \mathbf{b_i}, \mathbf{x}_{1:i-1}), \sigma_i(\mathbf{W}_i, \mathbf{b_i}, \mathbf{x}_{1:i-1})\right)$$

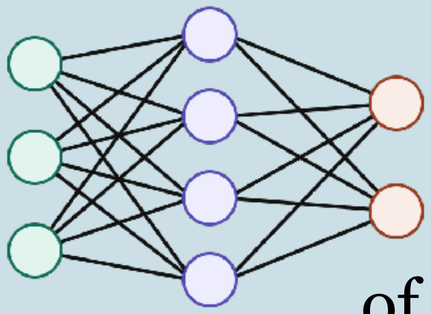
autoregressive density estimation — estimate some N-dim probability distribution $p(\mathbf{x})$

$$p(\mathbf{x}) = \prod_{i=1}^{N} p(x_i \mid \mathbf{x}_{i-1:1})$$

describe using a **neural network**

e.g.

$\mathrm{NN}(\mathbf{W}_i, \mathbf{b}_i, \mathbf{x}_{1:i-1})$



μ_i

$\log \sigma_i$

of 1D Gaussian

$p(x_i \mid \mathbf{x}_{1:i-1})$ is easy to sample

$$x_i' \sim \mathcal{N}\left(\mu_i(\mathbf{W}_i, \mathbf{b_i}, \mathbf{x}_{1:i-1}), \sigma_i(\mathbf{W}_i, \mathbf{b_i}, \mathbf{x}_{1:i-1})\right)$$

and evaluate $\log p(x_i \mid \mathbf{x}_{i-1:1}) = -\frac{(x_i - \mu_i)^2}{2\sigma^2} - \log \sigma_i + \mathrm{con}$

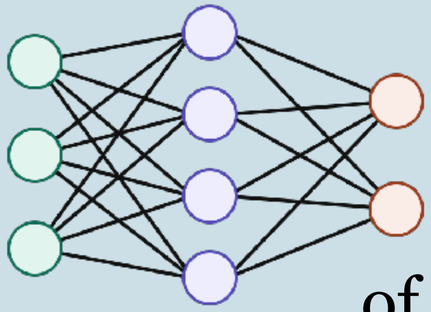
autoregressive density estimation — estimate some N-dim probability distribution $p(\mathbf{x})$

$$p(\mathbf{x}) = \prod_{i=1}^{N} p(x_i \mid \mathbf{x}_{i-1:1})$$

describe using a **neural network**

e.g.

$\text{NN}(\mathbf{W}_i, \mathbf{b}_i, \mathbf{x}_{1:i-1})$



of 1D Gaussian

$p(x_i \mid \mathbf{x}_{1:i-1})$ is easy to sample

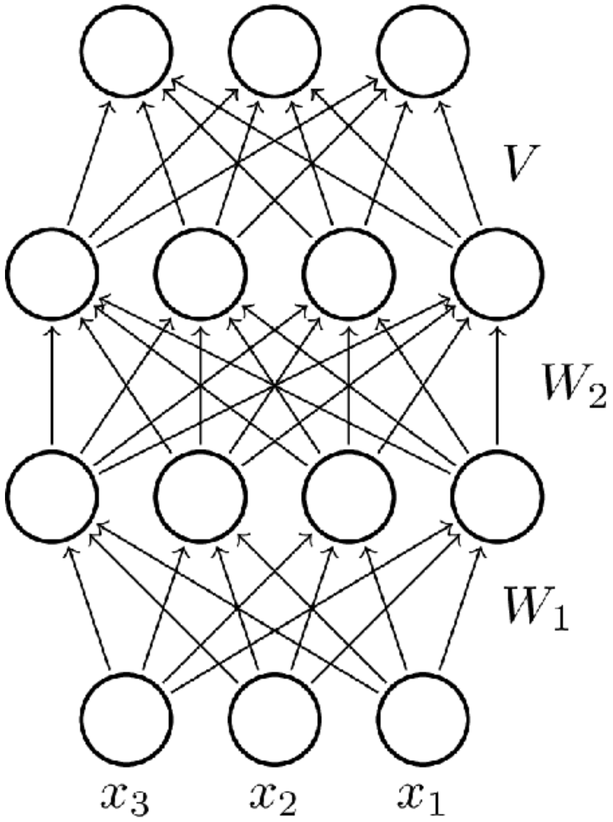
$$x_i' \sim \mathcal{N}\left(\mu_i(\mathbf{W}_i, \mathbf{b_i}, \mathbf{x}_{1:i-1}), \sigma_i(\mathbf{W}_i, \mathbf{b_i}, \mathbf{x}_{1:i-1})\right)$$

and evaluate $\log p(x_i \mid \mathbf{x}_{i-1:1}) = -\frac{(x_i - \mu_i)^2}{2\sigma^2} - \log \sigma_i + \text{con}$

$\Rightarrow p(\mathbf{x})$ **is easy to sample and evaluat**

masked autoencoder for distribution estimation (MADE) — *Germain et al.* (2015)

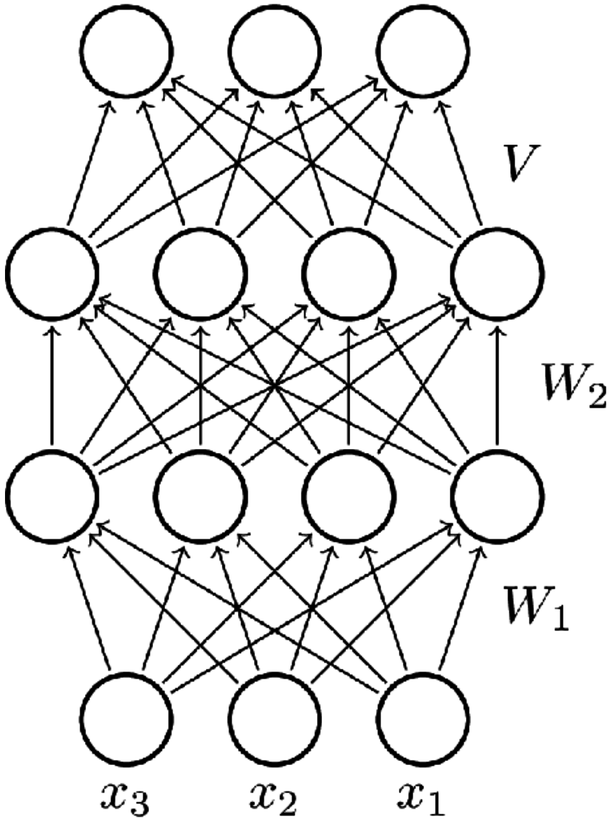
MADE



neural network with same input and output dimensions

**Germain et al.(2015) refer to this as an autoencoder*

MADE



$M_V =$

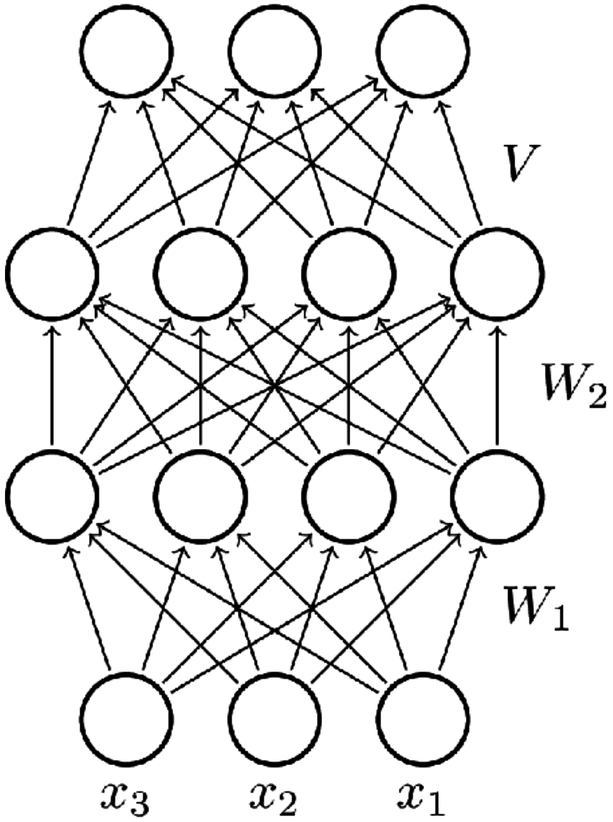
$M_{W_2} =$

$M_{W_1} =$

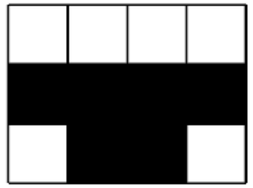
masks to enforce **autoregressive** property

$\times$ **masks**

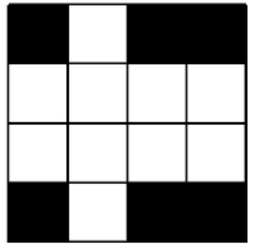
MADE



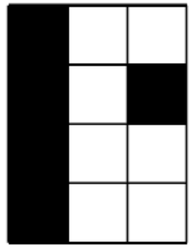
$M_V =$

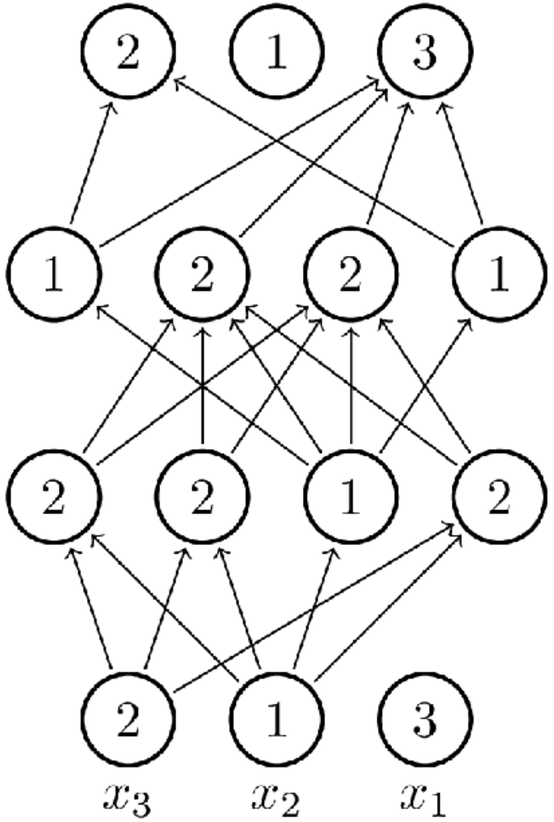


$M_{W_2} =$



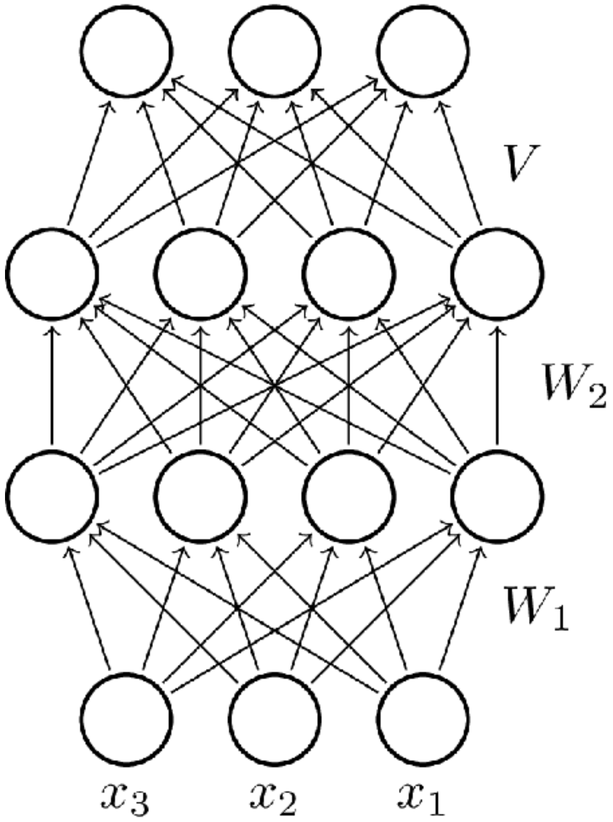
$M_{W_1} =$



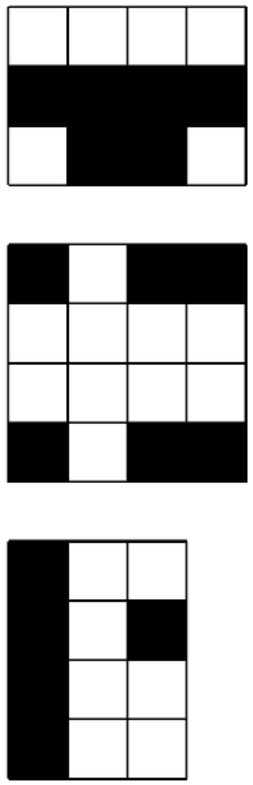


$\times$ **masks**

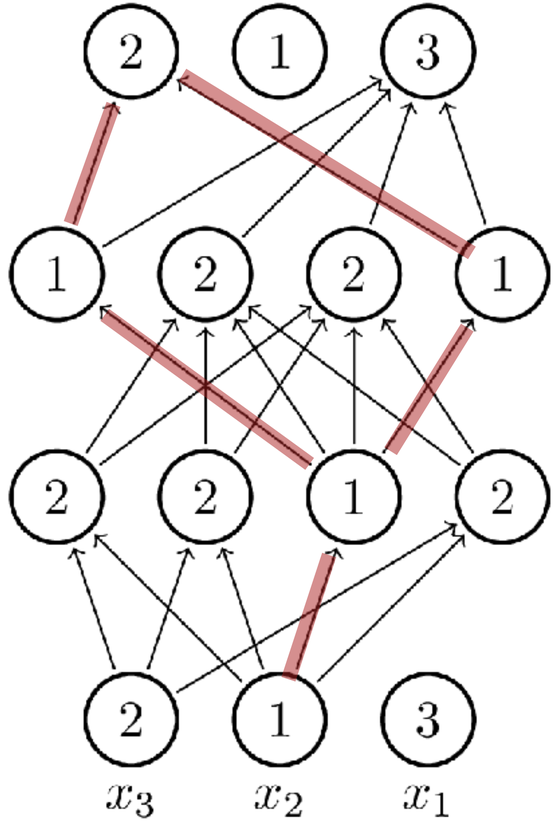
MADE



$$M_V = \quad M_{W_2} = \quad M_{W_1} =$$

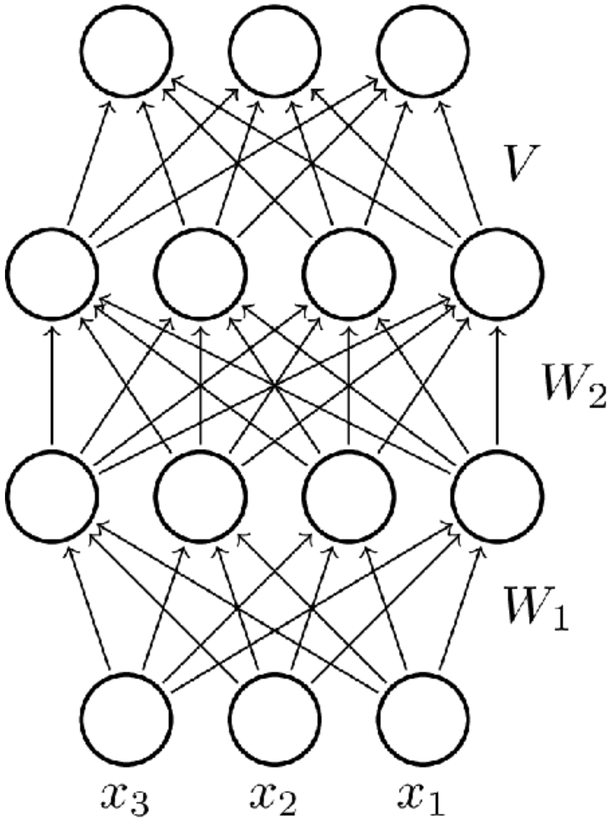


$p(x_3 \mid x_2)$

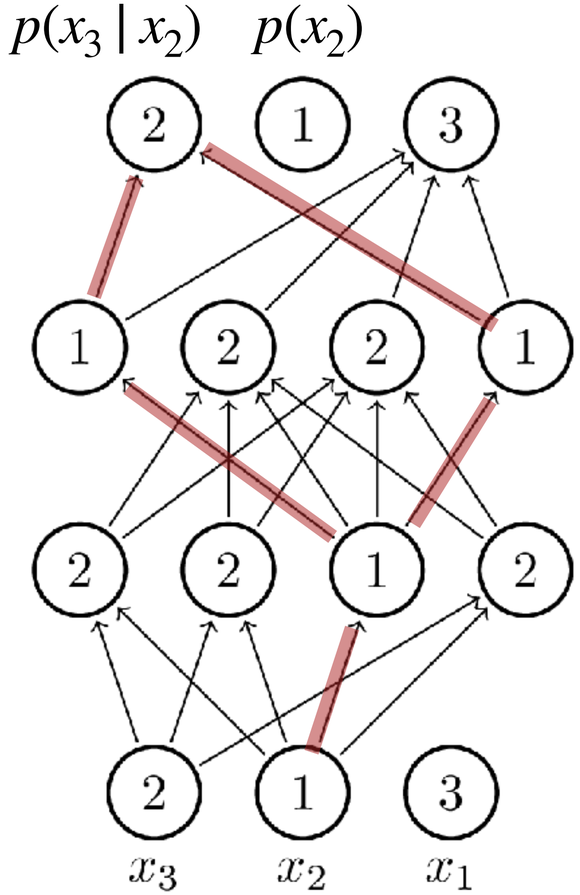


$\times$ **masks**

MADE

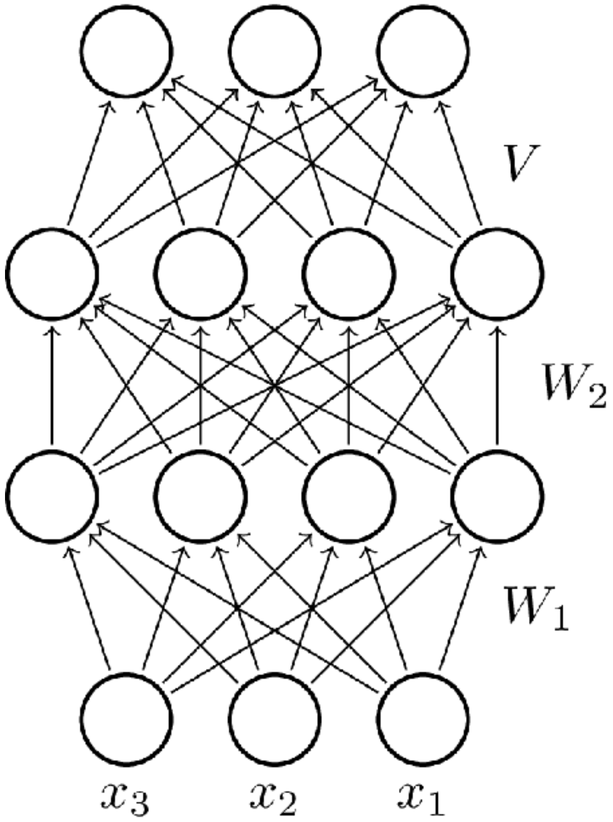


$$M_V = \quad M_{W_2} = \quad M_{W_1} =$$

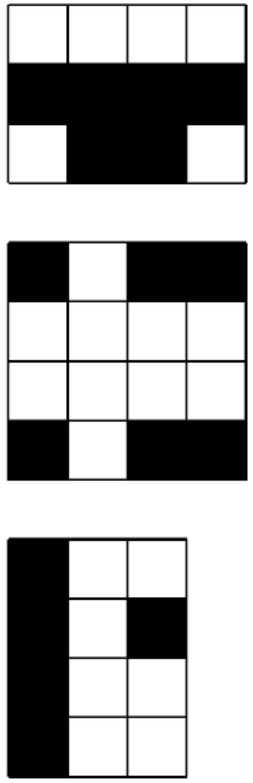


$\times$ **masks**

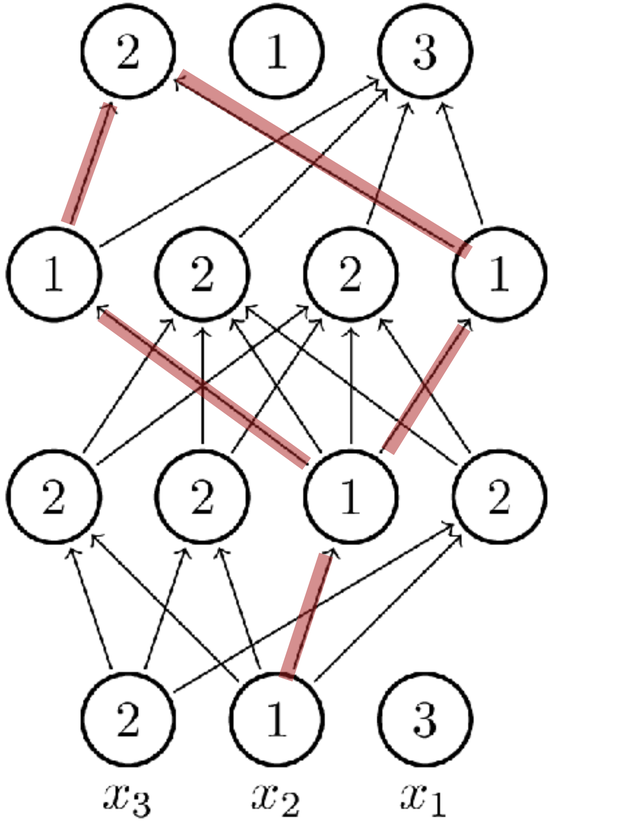
MADE



$$M_V = \quad M_{W_2} = \quad M_{W_1} =$$

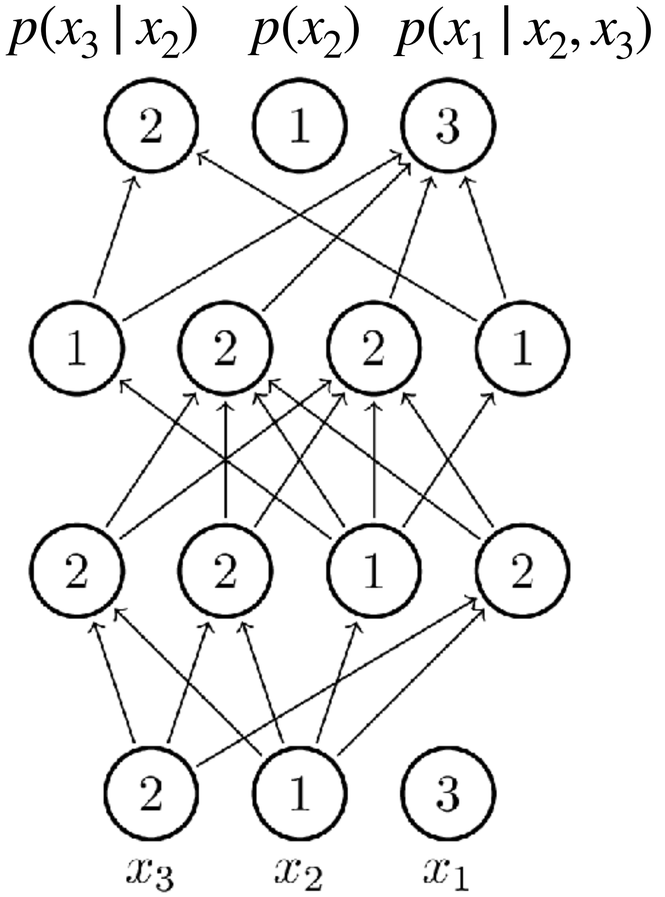


$p(x_3 \mid x_2) \quad p(x_2) \quad p(x_1 \mid x_2, x_3)$



$\times$ **masks**

MADE

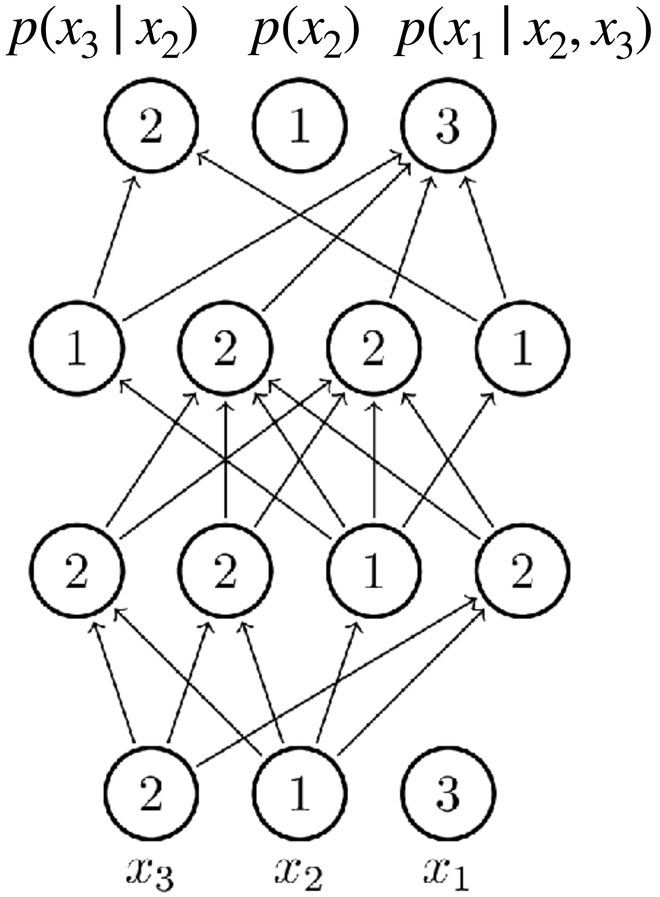


x

MADE

$$\mathbf{h}_1 = g(\mathbf{x}\,\mathbf{W}_1 \odot \mathbf{M}_1 + \mathbf{b}_1)$$

$$\mathbf{x}$$



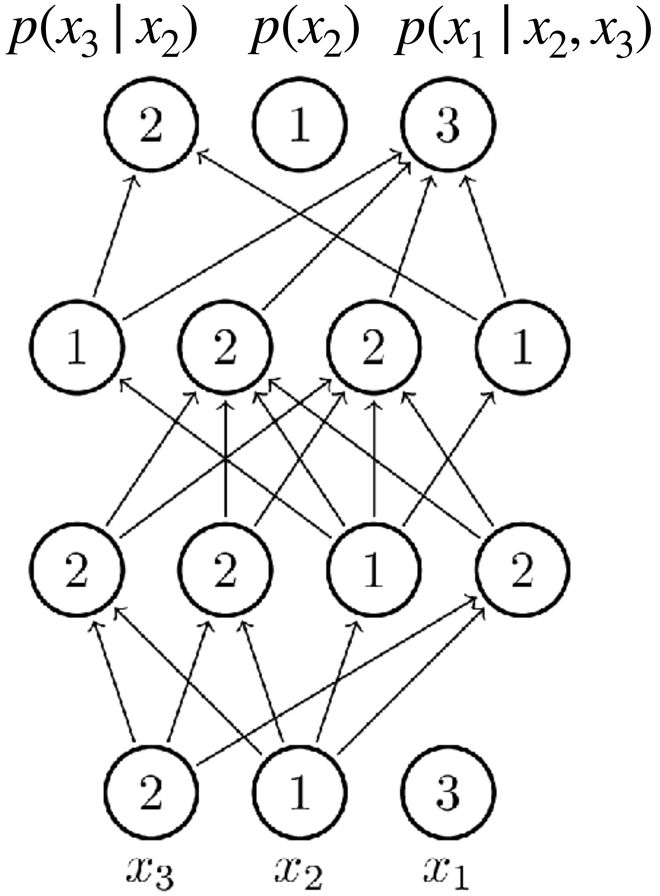
MADE

mask

$$\mathbf{h}_1 = g(\mathbf{x}\,\mathbf{W}_1 \odot \mathbf{M}_1 + \mathbf{b}_1)$$

activation function, weights, biases

$\mathbf{x}$

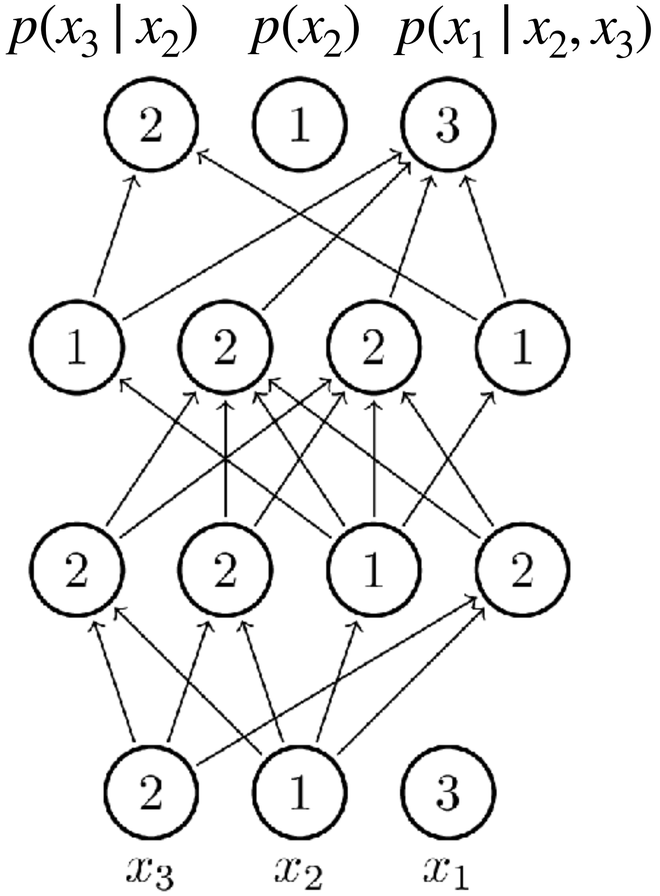


MADE

$$\mathbf{h}_2 = g(\mathbf{h}_1 \mathbf{W}_2 \odot \mathbf{M}_2 + \mathbf{b}_2)$$

$$\mathbf{h}_1 = g(\mathbf{x} \mathbf{W}_1 \odot \mathbf{M}_1 + \mathbf{b}_1)$$

$$\mathbf{x}$$



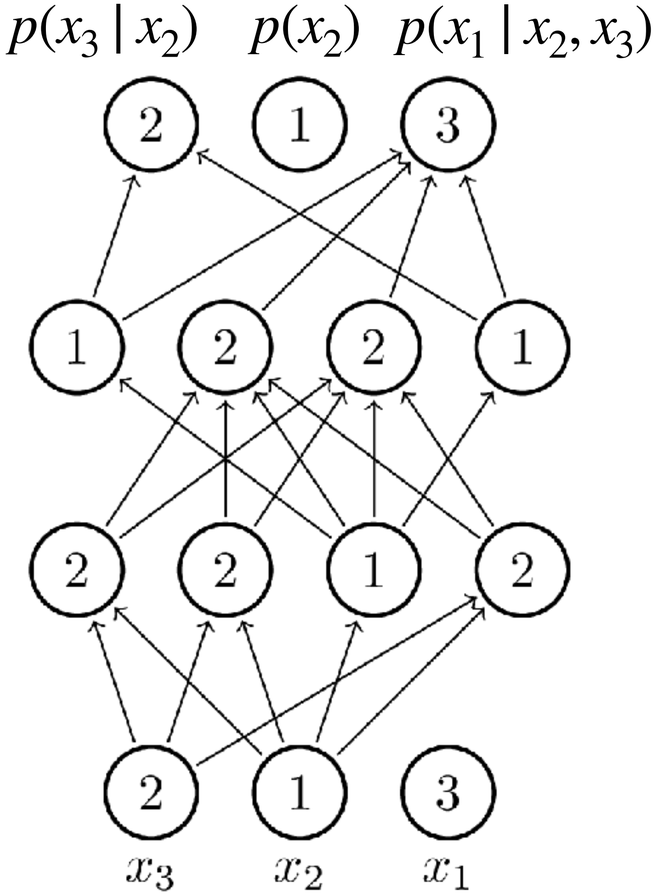
MADE

$$(\mu, \log \sigma) = \mathbf{h}_2 \mathbf{W}_3 \odot \mathbf{M}_3 + \mathbf{b}_3$$

$$\mathbf{h}_2 = g(\mathbf{h}_1 \mathbf{W}_2 \odot \mathbf{M}_2 + \mathbf{b}_2)$$

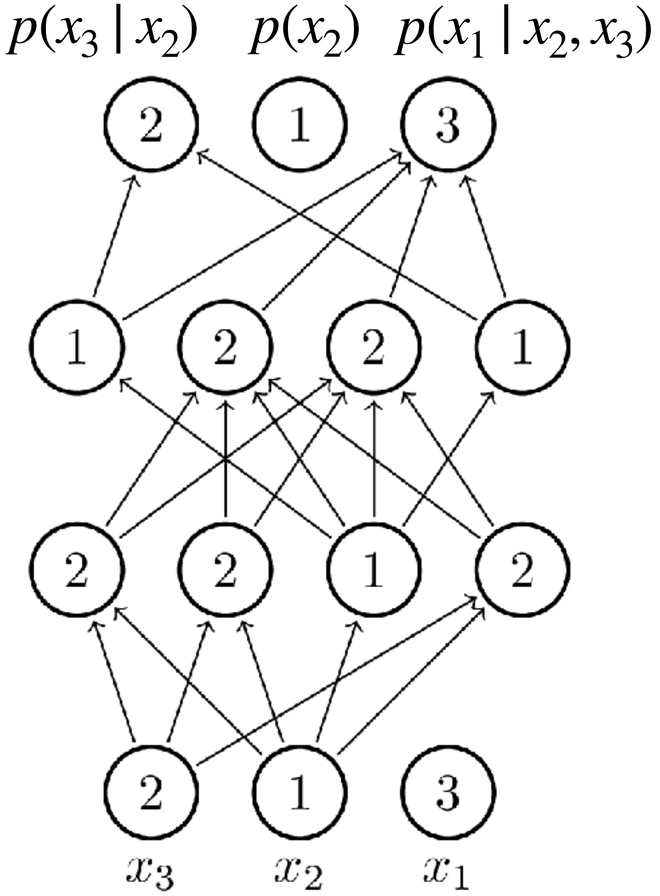
$$\mathbf{h}_1 = g(\mathbf{x} \mathbf{W}_1 \odot \mathbf{M}_1 + \mathbf{b}_1)$$

$$\mathbf{x}$$



MADE

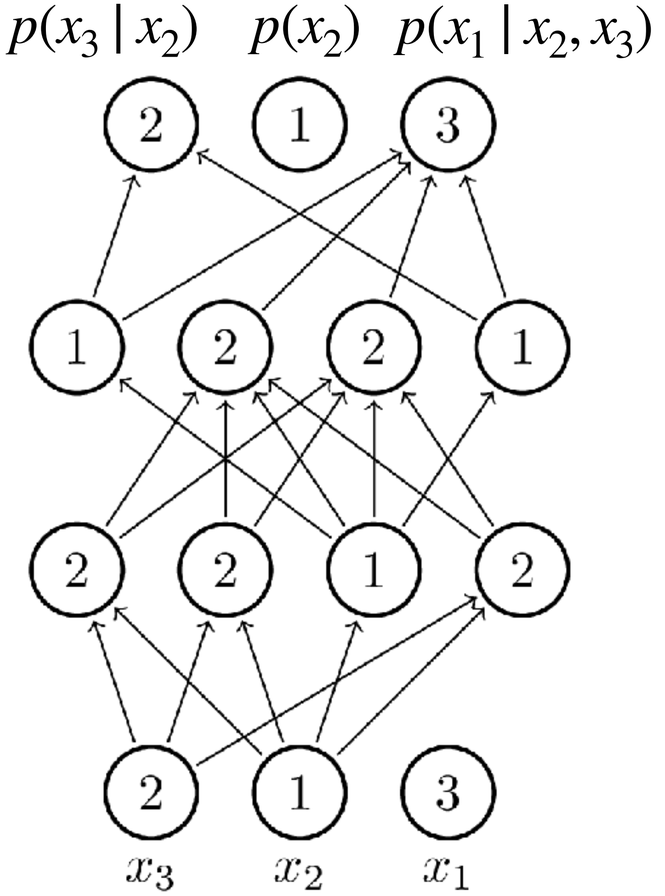
$$\min_{\phi} D_{\mathrm{KL}}\left(p(\mathbf{x}) \parallel q_{\phi}(\mathbf{x})\right) \approx \sum_{\mathbf{x}' \sim p(\mathbf{x})} \left[-\log q_{\phi}(\mathbf{x}')\right]$$



MADE

$$\min_{\phi} D_{\mathrm{KL}}\left(p(\mathbf{x}) \parallel q_{\phi}(\mathbf{x})\right)$$

$$\approx \sum_{\mathbf{x}' \sim p(\mathbf{x})} \left[-\log q_{\phi}(\mathbf{x}')\right]$$

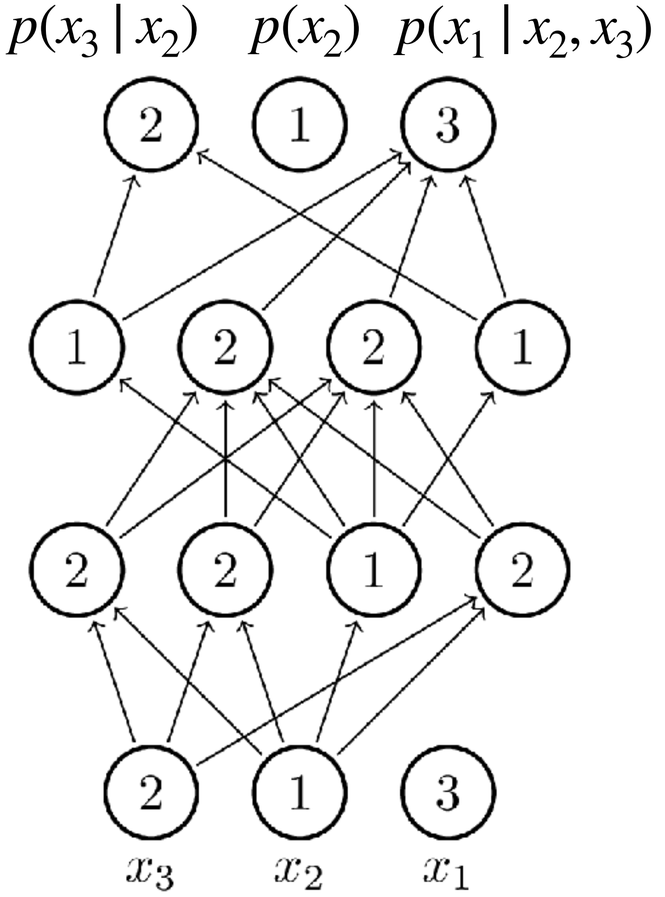


MADE

$$\min_{\phi} D_{\mathrm{KL}}\left(p(\mathbf{x}) \parallel q_{\phi}(\mathbf{x})\right)$$

$$\approx \sum_{\mathbf{x}' \sim p(\mathbf{x})} \left[-\log q_{\phi}(\mathbf{x}')\right]$$

$$= \sum_{\mathbf{x}' \sim p(\mathbf{x})} \left[-\sum_{i} \log q_{\phi}(x'_i \mid \mathbf{x}'_{i-1:1})\right]$$



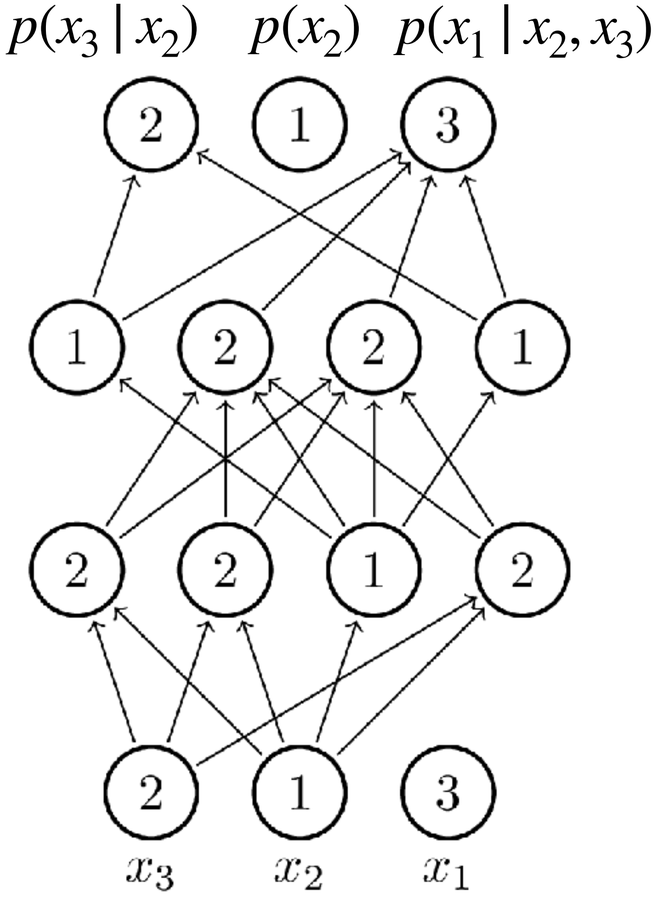
MADE

$$\min_{\phi} D_{\mathrm{KL}}\left(p(\mathbf{x}) \parallel q_{\phi}(\mathbf{x})\right)$$

$$\approx \sum_{\mathbf{x}' \sim p(\mathbf{x})} \left[-\log q_{\phi}(\mathbf{x}')\right]$$

$$= \sum_{\mathbf{x}' \sim p(\mathbf{x})} \left[-\sum_{i} \log q_{\phi}(x'_i \mid \mathbf{x}'_{i-1:1})\right]$$

$$= \sum_{\mathbf{x}' \sim p(\mathbf{x})} \left[-\sum_{i} \log \mathcal{N}(x'_i \mid \mu_i(\mathbf{x}'_{i-1:1}), \sigma_i^2(\mathbf{x}'_{i-1:1}))\right]$$



in-class: SBI with neural density estimation and MADE

ast384c/inclass/lec21_nde/nde_made.ipynb